QALight

Frontend-розробка

Заняття 7. Стандартна блокова модель

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Керування типом елемента, властивість display. Ширина та висота вікна перегляду, відступи, рамки, властивість box-sizing, управління потоком

Розробка унікального стилю для власного сайта.

Керування типом елемента

властивість display. Ширина та висота вікна перегляду, відступи, рамки

Правило !important

Теорія: правило !important

У CSS правило **!important** – це ключове слово, яке використовується для надання певному значенню властивості найвищого пріоритету, перевизначаючи всі інші конфліктуючі оголошення, незалежно від специфічності чи порядку в таблиці стилів.
Ключове слово !important додається в кінці оголошення CSS, безпосередньо перед крапкою з комою.

```
selector {
 property: value !important;
}
```

Відповідні випадки використання:
Перевизначення вбудованих стилів: !important - єдиний спосіб перевизначити вбудований стиль (встановлений в HTML за допомогою атрибута style) зовнішнім або вбудованим файлом стилів.
Робота зі сторонніми бібліотеками/CMS: Коли ви не можете редагувати оригінальні CSS-файли системи керування контентом або сторонньої бібліотеки, !important дозволяє застосувати власні стилі.
Доступність та таблиці стилів користувача: Його можна використовувати в таблицях стилів, визначених користувачем (для цілей доступності, таких як великі шрифти або певні комбінації кольорів), щоб забезпечити пріоритет користувацьких налаштувань над стилями автора.

Властивість display

Теорія: властивість display

Багатоцільова властивість **display** визначає, як елемент має відображатись у документі. Елемент відображається у вигляді блоку.

```
.nav-link {
   display: block;
}
```

Для вбудованих елементів, наприклад <img>, змушує його поводитись подібно блокам.

```
img {
   display: block;
}
```

Значення inline-block генерує **блоковий елемент**, що **обтікається іншими елементами веб-сторінки**. Такий елемент схожий на вбудовані елементи (<**img**>). Проте його внутрішня частина форматується як блоковий елемент, а сам елемент - як вбудований.

```
.nav-item {
   display: inline-block;
}
```

Блокова модель

Теорія: Блокова модель

Блокова модель CSS є основою структурування веб-сторінки - **кожний елемент** має прямокутну область вмісту, смугу відступів навколо вмісту, межу навколо заповнення та поле за межами. Браузер визначає, які стилі потрібно примінити до змісту кожного вікна.

Блокова модель (**box model**): https://www.w3.org/TR/css-box-3/ включає 4 області: **margin** (зовнішні поля), **border** (границі), **padding** (внутрішні поля), та **content** (вміст).

```
.product {
    display: block;
    margin: 1em;
    padding: 1em;
    border: 1px solid rgba(0, 0, 0, 0.125);
}
```

розмір кожної області визначається відповідними властивостями і може бути нульовим (або, у випадку полів, негативним).

Область вмісту

Теорія: Область вмісту

Область вмісту елемента (**content area**) містить текст та інші вкладені елементи. Часто має фон, колір або зображення (колір фону приховується непрозорим зображенням).

Розміри області вмісту - це **ширина контенту** (**content width чи content-box width**), та **висота контенту** (**content height чи content-box height**).

Кожний елемент має **ширину і висоту за замовчуванням,** які можуть дорівнювати нулю.

Ці властивості можна вказувати у будь-яких одиницях виміру CSS.

- **height** — висота;
- **min-height** — мінімальна висота;
- **max-height** — максимальна висота;
- **width** — ширина;
- **min-width** — мінімальна ширина;
- **max-width** — максимальна ширина.

Ширина елементу

Теорія: Блокова модель

За замовчуванням **ширина елементу ґрунтується на значенні display**.

Блокові елементи займають всю доступную ширину, що дорівнює ширині батьківського контейнера або вікна браузера.

inline та inline-block елементи розширюються та стискаються по горизонталі щоб розмістити їх вміст. inline **елементи не можуть мати фіксований розмір**, тобто, ширина та висота властиві **лише не inline елементам**. Визначити ширину для не inline елементів можна скориставшись властивістю **width**:

```
/* container */
.container {
   width: 100%;
}
```

Висота елементу

Теорія: Блокова модель

Висота елементу за замовчуванням визначається його вмістом. Елемент буде розширено чи стиснено по вертикалі при необхідності. Встановлюється висота для **не inline** елементів через **властивість height**:

```
.post { height: 250px;}
```

Якщо встановити блоковому елементу ширину чи висоту меншу, ніж вміст елемента, то вміст **випаде** з блоку.

```
.post {
    display: block;
    width: 300px;
    height: 250px;
}
```

Inline елементи

Практика: inline елементи

inline елементи не реагують на встановлення ширини та висоти. Встановити їх **можна лише блоку**:

```
.span {
  width: 100px;
  height: 100px;
  display: block;
}
```

Повернути значення за замовчуванням можна за допомогою значення **auto**:

```
.span {
  width: auto;
  height: auto;
}
```

Мінімальна і максимальна ширина

Практика: Мінімальна і максимальна ширина

Мінімальна і максимальна ширина **за замовчуванням** дорівнює **none**, проте ширина - **auto**, тобто **розраховується автоматично**. **Від'ємні значення для таких властивостей заборонені**.

ширина body 960px

```
body {
  width: 992px;
}
```

максимальная ширина container не більше 60% від body.

```
.container {
  min-width: 992px !important;
  width: 100%;
}
```

Властивість overflow

Практика: Властивість overflow

Властивість overflow керує тим, що відбувається з вмістом, який не поміщається в даній області.
Властивість overflow має такі значення:

- visible: значення за замовчуванням - властивість не встановлювати.
- hidden: переповнений вміст буде приховано
- scroll: встановлює смуги прокрутки, якщо потрібно
- auto: встановлює смуги прокрутки, навіть якщо вони не потрібні.

Властивість overflow встановлює бажану поведінку для переповнення елемента, тобто коли вміст елемента занадто великий, щоб поміститися в його контекст форматування блоку в обох напрямках.

Ця властивість є скороченням наступних властивостей CSS:

- overflow-x
- overflow-y

Альтернативний спосіб запобігти переповненню контейнера плаваючим елементом - встановити для властивості **`overflow`** контейнера прийнятне значення, **відмінне від visible.**

```
.div-with-img { overflow: auto; }
```

Практична робота

Створення footer

Створення footer

Практика: створення footer

```
<footer class="footer">
    <section class="footer-main">
        <div class="footer-main-item">
            <h3 class="footer-title">About</h3>
        </div>
        <div class="footer-main-item">
            <h3 class="footer-title">Contacts</h3>
        </div>
        <div class="footer-main-item">
            <h3 class="footer-title">Resources</h3>
        </div>
    </section>
```

Створення footer

Практика: створення footer

```
.footer {
        background-color: var(--bgColor);
        color: var(--muteColor);
        width: 100%;
        max-width: 100%;
}

.footer ul {
    list-style: none;
    padding-left: 0;
}

.footer a {
    color: var(--navColor);
}

.footer a:hover {
    color: var(--accentColor);
}
```

Секції footer-main-item

Практика: створення footer

```
<section class="footer-main">

    <div class="footer-main-item">
        <h3 class="footer-title">About</h3>
        <ul>
            <li><a href="index.html">Home</a></li>
            <li><a href="blog.html">Blog</a></li>
            <li><a href="about.html">About</a></li>
            <li><a href="contact.html">Contact</a></li>
        </ul>
```

Секції footer-main-item

Практика: створення footer

```
<div class="footer-main-item">
    <h3 class="footer-title">Contacts</h3>
    <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="blog.html">Blog</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="contact.html">Contact</a></li>
    </ul>
</div>
```

Секції footer-main-item

Практика: створення footer

```
        <div class="footer-main-item">
            <h3 class="footer-title">Resources</h3>
            <ul>
                <li><a href="index.html">Home</a></li>
                <li><a href="blog.html">Blog</a></li>
                <li><a href="about.html">About</a></li>
                <li><a href="contact.html">Contact</a></li>
            </ul>
        </div>

    </section>
```

Сукція footer-social

Практика: створення footer

```
<section class="footer-social">
    <div class="footer-social-list">
        <a href="#" class="fa-brands fa-facebook"></a>
        <a href="#" class="fa-brands fa-twitter"></a>
        <a href="#" class="fa-brands fa-google"></a>
        <a href="#" class="fa-brands fa-facebook"></a>
        <a href="#" class="fa-brands fa-twitter"></a>
        <a href="#" class="fa-brands fa-google"></a>
    </div>
</section>
```

Сукція footer-legal

Практика: створення footer

```
    <section class="footer-legal">
        <ul class="footer-legal-list">
            <li>
                <a href="#">Terms &amp;amp; Conditions</a>
            </li>
            <li>
                <a href="#">Privacy policy</a>
            </li>
            <li>
                &copy; 2026 My Website. All rights reserved.
            </li>
        </ul>

    </section>


</footer>
```

Властивість padding

Практика: Властивість padding

Властивість padding визначає відстань від контенту блока до його границі, а **margin** - відстань між границями сусідніх елементів.

Для внутрішніх полів наслідується колір фону елемента, на зовнішні - ні.

Внутрішні та зовнішні поля можна встановлювати у скороченій формі, або для кожної з сторін окремо.

Значення **для зовнішніх полів можуть бути від'ємними**, **для внутрішніх - ні**.

- padding/margin-top — верхнє поле (внутрішнє/зовнішнє)
- padding/margin-right — праве поле (внутрішнє/зовнішнє)
- padding/margin-bottom — нижнє поле (внутрішнє/зовнішнє)
- padding/margin-left — ліве поле (внутрішнє/зовнішнє)

Значення властивості padding

Практика: Властивість padding

Можна використовувати 1, 2, 3 чи 4 значень

1. Поля будут встановлені одночасно для кожної межі елемента. padding: 10px;
2. Перше значення встановлює поля від верхньої та нижньої межі, друге - від лівого та правого. padding: 5px 10px;
3. Перше значення встановлює поле від верхньої межі, друге - одночасно від лівої та правої межі, третє - від нижньої межі. padding: 5px 10px 15px;
4. Встановлює поля від верхньої, правої, нижньої та лівої межі.

Можна встановити поля для різних меж за допомогою властивостей **padding-left, padding-right, padding-top, padding-bottom**.

Властивість margin

Практика: Властивість margin

Властивість margin дозволяє встановити величину відступу **(margin area)** від кожної границі елемента. Відступ - це простір від границі поточного елемента до границі його батьківського елемента. margin не мають фону.

```
div { margin: 20px; }
```

Якщо елемент не має пращура, тоді margin - це відстань від межі елементу до межі вікна браузера. Вікно також має margin. Щоб позбутись його, потрібно встановити значення margin для селектора <body> у нуль.

```
body {  margin: 0;}
```

Властивість margin дозволяє встановити величину поля відразу для всіх сторін елемента або визначити її лише для вказаних сторін.

margin: [значение | проценты | auto] {1,4} | inherit

Значення властивості margin

Практика: Властивість margin

Margin охоплює блок та відділяє інші блоки на сторінці. Можна використовувати 1, 2, 3 або 4 значень, розділивши їх пробілом.

1. Поля будуть встановлені одночасно від кожної границі елемента.margin: 10px;
2. Перше значення встановлює поле зверху та знизу, друге - зліва та справа. margin: 5px 10px;
3. Перше значення встановлює поле зверху, друге - одночасно зліва та справа, а третє - знизу. margin: 5px 10px 15px;
4. Встановлюються поля зверху, справа, знизу, зліва. margin: 5px 10px 15px 20px;

Значення можуть бути від'ємними числами. auto - розмір поля розраховується автоматично браузером. Можна вказати поля для окремих сторін за допомогою margin-left, margin-right, margin-top, margin-bottom.

Поля двух сусідніх елементів, що розташовані один над одним або вкладені один в одний, могут об'єднуватись. Це називається **об'єднання полів** (**margin collapsing**).

Відстань між двома ***блоками*** дорівнює **більшому полю (margin), а не їх сумі**.

Об'єднуються лише вертикальні поля.

Для inline елементів margin працює лише по горизонталі - зліва та справа від елементів.

Inline елементи реагують тільки на горизонтальні поля.

Властивості padding і margin

Практика: Властивість margin

```
body {
    font-family: "Roboto", sans-serif;
    font-optical-sizing: auto;
    font-weight: 400;
    font-style: normal;
    font-stretch: 100%;
    font-size: 90%;
    font-variation-settings: "wdth" 100;
    margin: 0;
    padding: 0;
    max-width: 100%;
```

Властивості padding і margin

Практика: Властивість margin

```
.footer-main {
   padding: 1rem 2rem;=
}
.footer-social {
   padding: 1rem 2rem;
}

.footer-social-list a {
   margin: .5rem;
}

.footer-legal {
   padding: 1rem 2rem;
}
```

Сукція hero

Практика: створення hero

```
<section id="hero" class="hero bg-cover bg-center py-4 px-4"
        style="background-image: url(/images/bg-banner.jpg);" >
            <div class="container">
                <p>New Inspiration 2026 </p>
                <h1>20% off on new season </h1>
                <a href="" class="btn btn-hero" >Browse collection </a>
            </div>
  </section>
```

Сукція hero

Практика: створення hero

```
.bg-cover {
   background-color : antiquewhite ;
   background-size : cover;
}
.bg-center {
   background-position:top 700;
}
.bg-attachment {
   background-attachment : fixed;
}
```

Сукція hero

Практика: створення hero

```
.py-4 {
  padding-top: 2rem;
  padding-bottom: 2rem;
}

.py-8 {
  padding-top: 4rem;
  padding-bottom: 6rem;
}

.px-4 {
  padding-left: 2rem;
  padding-right: 2rem;
}
```

Область границі

Практика: Область границі

Область границі (border edge) розташована між зовнішнім полем padding та внутрішнім полем margin. За замовчуванням розмір границі (border) дорівнює 0, тобто її не видно.

Короткий запис властивості border дозволяє встановити всі основні параметри для всіх сторін, наприклад, **border: 1px solid black.**

Запис властивості border можна розділити на кілька різних властивостей:

- **border-top, border-right, border-bottom, border-left**: ширина, стиль та колір однієї з сторін границі.
- **border-width, border-style, border-color**: ширина, стиль або колір окремо, для всіх сторін границі.

Можна встановити одну з 3 властивостей для однієї з сторін окремо, за допомогою border-top-width, border-top-style, border-top-color, тощо.

Властивість border

Практика: властивість border

Універсальна властивість **border**: [border-width || border-style || border-color] | inherit дозволяє одночасно встановити товщину, стиль та колір границі навколо елементу. Значення можна вказати у будь-якому порядку, розділяючи їх пробілом

```
.btn { border: 1px solid transparent;}
```

Встановити границю можна за допомогою властивостей border-width, border-style, border-color:

```
.btn {
  border-width: 1px;
  border-style: solid;
  border-color: transparent;
}
```

Ширину border можна встановити за допомогою **border-width** або у складі властивості **border**. Значення **border-width** вказують або за допомогою позитивних значень, або за допомогою ключових слів **thin, medium, thick**.

Властивість border-style

Практика: властивість border

Border-style управляє виглядом **border**.

border-style: [none | hidden | dotted | dashed | solid | double | groove | ridge | inset | outset] {1,4} | inherit

Встановлює стиль границі елемента. Можна задавати індивідуальні стилі для різних сторін елемента. Вигляд залежить від браузера. Найпоширеніші значення стилів:

- solid — неперервна;
- dashed — пунктир;
- dotted — крапками.

Можливо використовувати 1, 2, 3 або 4 значення.

1. Стиль границі буде задано для всіх сторін елемента.
2. Перше значення встановлює стиль верхньої і нижньої границі, друге - лівої та правої.
3. Перше значення - стиль верхньої границі, друге - лівої і правої границь, третє - нижньої.
4. Встановлюється стиль верхньої, правої, нижньої та лівої границь.

None - товщина границі (border-width) дорівнює нулю. Hidden - товщина границі дорівнює нулю та властивість border-collapse дорівнює collapse.

Границі

Практика: Границі

Границі можуть бути розташовані з однієї сторони елемента: border-top, border-right, border-bottom и border-left.

```
div {
  border-bottom: 6px solid #949599;
}
```

Наприклад, якщо потрібно змінити лише ширину нижньої границі:

```
div {
  border-bottom-width: 12px;
}
```

Властивість border-radius

Практика: Властивість border-radius

Властивість border-radius визначає радіус скруглення кутів елемента. Одне значення заокруглює всі 4 кута елемента; 2 значення - лівий верхній/правий нижній та правий верхній/лівий нижній кути; 4 значення заокруглюють лівий верхній, правий верхній, правий нижній та лівий нижній кути.

Властивість border-radius можна встановити для окремих кутів елемента.

```
.btn {   border-top-right-radius: 5px; }
```

Радіус застосовується до всіх background, навіть якщо елемент не має границі;

Значення за замовчуванням:

```
border-top-left-radius: 0; border-top-right-radius: 0; border-bottom-right-radius: 0;    border-bottom-left-radius: 0;
```

Властивість box-sizing

Практика: Властивість box-sizing

За замовчуванням, якщо CSS-властивість **box-sizing** не вказана, розмір блоку визначається властивостями **width, min-width, max-width, height, min-height** та **max-height**.

Існує 2 блочні моделі: **модель W3C**, що підтримується всіма основними браузерами та **традиційна** - ii підтримує IE6 у режимі зворотної сумісності. Структура блоку елемента абсолютно однакова. Відмінність моделей - це правила визначення розмірів елемента.

Властивість box-sizing використовується для зміни **блочної моделі CSS**, за допомогою якої обчислюють ширину та довжину елементів.

box-sizing: **content-box;**
У блочній моделі W3C розміри елемента включають лише контентну частину. Тобто width і height не включають а ні розміри внутрішніх відступів, а ні розміри зовнішніх полів

box-sizing: **border-box;**

Властивості **width і height обчислюються як сума контент, padding та border**, проте **не включають margin**.

Загальна ширина і висота

Практика: Загальна ширина і висота

У традиційній моделі елемент має такі ж розміри, які визначені у CSS.

У моделі W3C визначені у CSS розміри (width і height) та дійсні розміри елемента не співпадають.

Загальну ширину елемента можна розрахувати за формулою:

margin-right + border-right + padding-right + width + padding-left + border-left + margin-left

Загальну висоту елемента можна розрахувати за формулою:

margin-top + border-top + padding-top + height + padding-bottom + border-bottom + margin-bottom

Псевдокласи

Практика: Псевдокласи

Псевдокласи - це спеціальні класи, які використовуються для визначення поведінки елементів HTML та для додавання спеціальних ефектів до деяких селекторів, які автоматично застосовуються в певних станах. Псевдоклас - це фантомний стан або специфічна характеристика елемента, на який можна орієнтуватися за допомогою CSS.
Кілька поширених псевдокласів: :link, :visited, :hover, :active, :first-child і :nth-child.

Перед назвами псевдокласів завжди стоїть двокрапка (**:**). Потім йде назва псевдокласу, а іноді й значення в дужках. Псевдокласи представляють стани, в яких може перебувати елемент:

- :active - Застосовується, коли користувач активує посилання (наприклад, натискаючи).
- :hover - Застосовується, коли користувач наводить курсор на посилання.
- :link - Застосовується до посилань, які користувач ще не відвідав.
- :visited - Застосовується до посилань, які користувач уже відвідав.

Усі вони представляють різні стани, в яких може перебувати посилання. Вони націлені на існуючі елементи, які перебувають у певному стані.

Псевдоклас :root

Псевдоклас :root відноситься до найвищого батьківського елемента в документі. Практично у всіх випадках псевдоклас :root посилатиметься на елемент html у документі HTML. Однак він може посилатися на інший елемент, якщо він використовується в іншій мові розмітки, наприклад SVG або XML.

Додамо колір фону до елемента html, найвищого батьківського елемента в документі HTML:

```
:root {
  background: orange;
}
```

Ми могли б досягти того ж ефекту, якби використовували html як селектор. Однак :root, як і клас, має вищу специфічність, ніж селектор елементів (html).

Псевдоклас :root

Практика: Псевдокласи

```
:root {
        --linkColor : rgba(9, 89, 9, .7);
        --navColor : hsl(0, 0%, 90%);
        --textColor : hsl(0, 0%, 25%);
        --muteColor : hsl(0, 0%, 95%);
        --bgColor: hsl(0, 0%, 22%);
        --accentColor : hsl(300, 50%, 50%);
    }

.nav-item a {
        color: white;
        text-decoration: none;
}
.nav-item a:hover {
    color: var(--accentColor);
    text-decoration: underline;
}
```

ВПРАВИ

Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Vs Code : https://code.visualstudio.com/

Webstorm: https://www.jetbrains.com/webstorm/

Web technology For developers https://developer.mozilla.org/en-US/docs/Web

CSS: https://developer.mozilla.org/en-US/docs/Web/CSS

Консорціум Всесвітнього павутиння https://www.w3.org/

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua